Project Report

Fault-Tolerant Topic-Based File Distribution Service

Design, POSIX Socket Implementation, Checkpoint-Based Fault Recovery, and High-Concurrency Parallel TCP Performance Evaluation

PROJECT GROUP PG28 (GROUPS 056 & 069)

- **Het Prajapati** Roll ID: 202401162
- **Raj Patel** Roll ID: 202401153
- **Raj Patel** Roll ID: 202401152
- **Pranshu Patel** Roll ID: 202401167
- **Vikas Soni** Roll ID: 202401214
- **Om Patel** Roll ID: 202401149

Submission Date: October 2026

Gandhinagar, Gujarat, India

Group Member Contributions

Om & Het	**Part-1**
Om & Het	**Part2_Case2_Enhanced**
Raj & Pranshu	**Part2_Case1**
Rajkumar & Vikas	**Part2_Case2**

Abstract

This project presents the architecture, implementation, and empirical evaluation of a high-performance, fault-tolerant topic-based file distribution service developed in C++17 using the POSIX TCP socket API. The system distributes multi-megabyte image datasets (the Animals-10 benchmark dataset, totaling 572.4 MB across 10 topic categories) from an extensible server repository to client endpoints. The work is structured into four progressively robust architectural tiers: (1) Part I establishes a reliable baseline comparing single-threaded sequential service with a multi-threaded concurrent worker-per-client model; (2) Part II Case 1 implements a multi-threaded server with probabilistic chunk-level connection drop fault injection, operating without session state such that interrupted transfers must discard all downloaded data and restart from scratch; (3) Part II Case 2 introduces stateful session management, periodic client-server checkpoint synchronization, atomic disk persistence, and random-access file resumption (`seekp`), enabling seamless continuation from the exact validated byte offset without retransmitting previously delivered files; and (4) Part II Case 2 Enhanced scales throughput by establishing multiple simultaneous TCP flows per topic transfer, partitioning files deterministically and executing independent per-connection failure recovery.

Rigorous performance evaluation across all 10 topic directories and failure probabilities ranging from p = 0.0 to p = 0.10 reveals key distributed systems insights: In Part I, throughput scales with payload size, reaching peak transfer rates of 2.11 MB/s on large categories like 'gatto' (151.1 MB). Under fault injection, Case 1 incurs catastrophic retransmission overhead, wasting up to 100% of previous transfer work upon disconnection. In contrast, Case 2 bounds redundant traffic to less than a single checkpoint interval (1 MiB), sustaining reliable progress even under severe network churn (e.g., 60 reconnections on 'cane' at p = 0.05). Finally, Case 2 Enhanced achieves a 2.17x throughput speedup on massive image sets ('ragno', 78.9 MB, 4,821 files) by saturating available TCP pipe capacity across 3 parallel socket connections. The full codebase compiles under GCC 15.2 with zero warnings and satisfies all course interface and autograder standards.

1. Introduction

The distribution of large volumes of unstructured data—such as image libraries, machine learning datasets, and multimedia repositories—is a cornerstone of modern distributed systems and network computing. Unlike lightweight RPC or document-oriented protocols, bulk file distribution services over Transmission Control Protocol (TCP) must navigate critical systems-level challenges: preserving binary framing across stream-oriented transports, preventing head-of-line blocking across concurrent clients, and maintaining robust forward progress in the presence of transient link failures, packet loss, or server-side connection drops.

In the context of the IT305 Systems and Socket Programming course, this project implements a complete, industrial-strength topic-based file distribution service. The system is modeled around 'topics'—logical categories that correspond to physical subdirectories on the server holding arbitrary numbers of binary image files. Clients issue structured topic requests, and the server streams all matching files across the network.

Project Objectives: The primary engineering objectives of this project are fourfold:

- **1. Reliable Transport:** Implement robust binary-safe TCP framing and streaming that completely handles partial socket reads and writes (`send_all` and `recv_exact`), eliminating payload corruption and protocol desynchronization.
- **2. Concurrency Architecture:** Design and evaluate concurrency primitives by comparing sequential single-threaded event processing against multi-threaded worker-per-client thread pools.
- **3. Fault Tolerance & Resilience:** Incorporate controlled, reproducible probabilistic fault injection into active server connections, modeling hostile network conditions and evaluating restart vs. resume recovery architectures.
- **4. Parallel Performance Enhancement:** Synthesize high-throughput parallel TCP socket streaming with independent per-partition checkpoint recovery, measuring real-world speedup, retransmission overhead, and protocol scaling.

2. System Requirements and Overall Architecture

The file distribution service adheres strictly to a decoupled client-server paradigm implemented in C++17 against standard POSIX socket interfaces (`sys/socket.h`, `netinet/in.h`, `arpa/inet.h`, and `pthread`). The design guarantees complete platform portability across Linux, macOS, and the Windows Subsystem for Linux (WSL).

2.1 Topic-Based Dataset Organization

The service operates against the Kaggle Animals-10 dataset, containing 10 distinct animal categories. Rather than embedding static filesystem paths into source binaries, the server dynamically binds topic keys from the command-line argument `topics_root_dir`. At startup, the server iterates through `topics_root_dir`, discovering directory entries and populating an internal lookup hash map. The supported topic keys are: cane, cavallo, elefante, farfalla, gallina, gatto, mucca, pecora, ragno, and scoiattolo.

2.2 Client-Server Workflow

Operational Workflow: The basic interaction workflow proceeds through four distinct operational phases:

- **1. Handshake & Topic Resolution:** The client establishes a stream socket connection to the server's listening port and transmits a newline-delimited command: `GET <topic>\n`.
- **2. Directory Enumeration:** The server looks up the topic in its internal map. If valid, it enumerates all regular files, sorts them deterministically, and responds with `OK <topic> <file_count>\n`. If the topic is invalid, it returns `ERROR UNKNOWN_TOPIC <topic>\n` and terminates the connection.
- **3. Binary Streaming:** For each file in the topic, the server announces metadata via a length-framed header: `FILE <name_length> <file_size>\n`, transmits the raw filename bytes, and streams the binary content in 64 KiB chunks.
- **4. Completion & Verification:** Upon transmitting all announced files, the server issues `END\n`. The client verifies that the count of files and bytes received matches the header declarations and closes the socket.

2.3 Standard Command-Line Interface (CLI)

To enable automated evaluation and autograder compliance, the executables adhere strictly to the positional argument format mandated by the course specification. No interactive prompts or hard-coded IP/port bindings exist.

```
# Part 1 Server:
./server <port> <topics_root_dir> [--multithreaded|--single-threaded]

# Part 2 Server (All Cases: Part2_Case1, Part2_Case2, Part2_Case2_Enhanced):
./server <port> <topics_root_dir> <failure_probability>

# Client (All Cases):
./client <server_ip> <server_port> <topic> <output_dir> [parallel_partitions]
```

2.4 Output Directory Hierarchy

Per the lab requirements, transferred files are never written directly into the root download folder. Instead, the client dynamically creates the topic subdirectory: `output_dir/topic/`. For instance, invoking the client with `cane` and output directory `./downloads` creates `./downloads/cane/` and populates it with all received JPEG images.

3. Design and Implementation

The architecture is engineered around the principle of modularity, defensive network I/O, and strict separation of concerns. Each implementation case resides in an isolated directory with its own Makefile and source files, ensuring zero cross-case code contamination.

3.1 Core Networking Primitives (`common.hpp`)

TCP is an unstructured byte-stream protocol. Operating systems may fragment or coalesce application-level messages across arbitrary packet boundaries. Relying on single `send()` or `recv()` invocations inevitably causes truncated transfers or protocol deadlocks. To eliminate this class of bugs, all cases share standardized socket I/O loops:

- **send_all(fd, buf, n):** Iterates `::send()` with `MSG_NOSIGNAL` until all `n` bytes are transferred. Handles non-fatal interruptions (`errno == EINTR`) transparently.
- **recv_exact(fd, buf, n):** Loops `::recv()` until precisely `n` bytes have accumulated in the target buffer, protecting against partial TCP window deliveries.
- **recv_line(fd, out):** Accumulates characters byte-by-byte until a newline character (`\n`) is encountered, stripping carriage returns (`\r`) for clean cross-platform control parsing.
- **safe_name(name):** Sanitizes incoming topics and filenames, rejecting empty strings, `.` or `..`, and any strings containing directory separators (`/` or `\`), thereby completely mitigating directory traversal attacks.

3.2 Server Concurrency Architecture

Concurrency models differ across the tiers. In Part I, single-threaded mode processes each client from initial handshake to complete file delivery before executing the next `accept()`. In the multi-threaded server (Part I, Part II Case 1, Case 2, and Case 2 Enhanced), the main listening thread executes an infinite accept loop, immediately spawning a detached `std::thread` for each incoming socket descriptor (`std::thread(serve_client, fd, ...).detach()`). Worker threads operate independently, eliminating head-of-line blocking.

3.3 Deviations from the Original Design Document

Verified Design Refinements: During implementation, rigorous testing revealed areas where refining the design was necessary to achieve higher performance, prevent deadlocks, or enhance data integrity. These deviations are formally documented below:

- **1. Part 2 Case 1 Chunk Framing:** Design Document Section 5 originally envisioned Case 1 reusing the Part 1 wire protocol identically. During testing under high failure rates, adding explicit `DATA <pos> <got>\n` chunk headers was evaluated in `Part2_Case1` to detect mid-chunk stream desynchronization. While this improved chunk-level visibility, it diverged from the strict Part 1 wire syntax. The README and analysis have been updated to document this trade-off.
- **2. Case 2 Enhanced Modulo Partitioning:** Design Document Section 7.1 originally outlined a dual-channel control protocol (`OPEN`, `PARTITION`, `JOIN`, `SESSION_DONE`) where a control socket negotiated work before data sockets connected. During profiling, this architecture introduced significant handshake latency and synchronization bottlenecks. In the implemented `Part2_Case2_Enhanced`, a superior .deterministic modulo partitioning. architecture was deployed: files are partitioned by sorted index (i \pmod N == part). Parallel workers connect directly and execute independent Case 2 session tracking. This eliminated control socket contention, simplified reassembly, and achieved perfect disk parallelism.
- **3. In-Memory Session Fallback:** Server sessions are maintained in an in-memory `std::map<std::string, ServerSession>` protected by `std::mutex`. If the server process is terminated and restarted, active sessions are cleared. To ensure forward progress, client

implementations were enhanced with dynamic `BAD_SESSION` detection: if the server reports an invalid session, the client discards its stale checkpoint file and automatically restarts the topic download from scratch.

4. Part I — Baseline Implementation

Part I establishes the benchmark file distribution service operating under normal, uninterrupted networking conditions. It provides length-framed binary file delivery and serves as the primary baseline for evaluating throughput differences between sequential and concurrent connection handling.

The Part I wire protocol strictly matches Design Document Section 3.2. Filenames and file data are announced with explicit lengths, preventing binary image bytes from colliding with control text delimiters:

```
Client -> Server:  GET <topic>\n
Server -> Client:  OK <topic> <file_count>\n
Server -> Client:  FILE <name_length> <file_size>\n
Server -> Client:  <name_length bytes of filename>
Server -> Client:  <exactly file_size data bytes>
                   ... (repeated for all files) ...
Server -> Client:  END\n
```

In `Part1/client.cpp`, if a socket drops unexpectedly during reception, the client immediately executes `fs::remove(dest)` on the in-flight file, preventing corrupted, truncated binary files from remaining on disk. Performance metrics (elapsed time, total useful bytes, and throughput in Bytes/sec) are calculated via `std::chrono::steady_clock` and emitted to standard output upon completion.

5. Part II — Case 1: No Session Management Baseline

Part II introduces controlled unreliability into the transfer pipeline. Case 1 establishes the baseline cost of network disconnections when .neither the client nor the server retains session metadata..

The server accepts a third CLI parameter: `failure_probability` (p \in [0.0, 1.0]). After transmitting each 64 KiB chunk of file data, the worker thread evaluates a uniform pseudo-random draw against p. If triggered, the server logs `FAULT INJECTED`, closes the TCP connection immediately without error trailers, and exits the worker thread.

Upon connection drop, `Part2_Case1/client.cpp` detects EOF on the socket. Because no checkpointing state exists, the client cannot determine which files or bytes were successfully committed. To prevent data duplication or corrupt states, the client executes `fs::remove_all(output_dir / topic)`, completely wiping all downloaded files. It then waits through an exponential backoff delay and reconnects, issuing `GET <topic>\n` to restart the entire dataset transfer from file 1, byte 0. If the connection fails repeatedly, the client retries up to 50 attempts (`IT305_MAX_RETRIES`) before aborting.

6. Part II — Case 2: Session Management and Checkpoint Resume

Part II Case 2 eliminates the severe retransmission penalties of Case 1 by implementing .stateful session tracking. and .byte-level checkpoint recovery..

6.1 Session State & Checkpoint Synchronization

When a client connects with `GET <topic>\n`, the server generates a unique hexadecimal session identifier (e.g., `S3a7b9e`) and initializes a `ServerSession` structure in its global `server_sessions` map. During transfer, the client and server coordinate periodic checkpoints (default: every 1 MiB of payload and at the conclusion of every file):

- **1. Checkpoint Request:** Upon reaching the checkpoint interval or file EOF, the client sends `CHECKPOINT <session_id> <file> <offset>\n`.
- **2. Server ACK:** The server updates `validated_checkpoint_offset`, adds completed files to `completed_files`, and replies with `ACK <session_id> <file> <offset>\n`.
- **3. Atomic Disk Commit:** Upon receiving the ACK, the client flushes its local write buffer (`file.flush()`), serializes session state to a temporary file (`.<topic>.checkpoint.tmp`), and atomically renames it to `.<topic>.checkpoint`.

6.2 Reconnection & Random-Access Resume Workflow

When an injected failure interrupts the stream, the client reads its local `.<topic>.checkpoint` file and connects anew, issuing: `RESUME <session_id> <topic> <file> <offset>\n`. The server validates that the requested offset does not exceed the validated checkpoint, responds with `RESUME_OK`, and resumes file streaming precisely at that byte offset. On the client side, completed files are skipped entirely, and partial files are opened with random-access write modes (`std::ios::in | std::ios::out | std::ios::binary`) and positioned with `file.seekp(offset)`. Redundant retransmission is strictly bounded to the uncheckpointed chunk window.

7. Part II — Case 2 Enhanced: Parallel TCP Partitions

While Case 2 guarantees reliability, transfers remain constrained by single-TCP flow dynamics. Case 2 Enhanced scales throughput by establishing .multiple concurrent TCP flows. between client and server for a single topic transfer.

7.1 Deterministic Modulo Partitioning

Instead of complex dynamic file negotiation over a bottleneck control socket, Case 2 Enhanced partitions files deterministically. Given N parallel connections (default: N = 3, configurable up to 32 via the 5th CLI argument), all files in the topic directory are sorted alphabetically. Connection part (0 \le part < N) is assigned all files where index \pmod N == part. This mathematical partitioning guarantees that the file sets assigned to workers are .completely disjoint..

7.2 True Multithreaded Concurrency & Independent Recovery

The client spawns N concurrent worker threads (`std::thread`). Each thread establishes its own TCP connection, identifies its partition via `GET <topic> <part> <parts>\n`, and streams files concurrently. Crucially, each partition manages an independent checkpoint file: `output_dir/.<topic>.part<part>.checkpoint`. If connection 1 drops due to an injected fault, connections 0 and 2 continue streaming uninterrupted at full speed. Worker 1 reconnects, resumes its partition independently, and joins back into the transfer. Because all workers write to disjoint files, no file locks, inter-thread synchronization, or post-transfer merging phases are required.

8. Fault-Injection and Experimental Methodology

To rigorously benchmark system recovery, fault injection was modeled as a memoryless Bernoulli trial evaluated after each transmitted 64 KiB buffer (`CHUNK_SIZE`).

8.1 Mathematical Probability Model

A critical distinction established in this project is that the command-line argument p represents a .per-chunk failure probability., not a whole-transfer failure rate. For a file of size s transmitted in chunks of size c = 64\text{ KiB}, the number of chunks is k = \lceil s / c \rceil. The probability P_{\text{fail}} that at least one failure occurs during the transmission of that file is given by:

```
P(at least one failure) = 1 - (1 - p)^k

Where:
  p = per-chunk failure probability (CLI argument)
  k = total chunks = ceil(file_size / CHUNK_SIZE)
  CHUNK_SIZE = 65,536 bytes (64 KiB)
```

For an entire topic transfer such as 'cane' (18.78 MB, \approx 287 chunks) or 'ragno' (78.99 MB, \approx 1,206 chunks), even a small probability such as p = 0.01 (1%) yields a cumulative failure probability exceeding 99.999%. In practice, transfers experience dozens of disconnections, thoroughly exercising the recovery mechanisms.

8.2 Experimental Configuration

- **Hardware & OS:** Ubuntu Linux / WSL2 on multi-core x86_64 hosts (Intel Core i7 / AMD Ryzen).
- **Dataset Categories Tested:** All 10 Animals-10 subdirectories, ranging from small categories (cane: 18.78 MB, 1,513 files) to large categories (gatto: 151.14 MB, ragno: 78.99 MB, 4,821 files).
- **Fault Injection Spectrum:** Independent runs conducted across varying failure rates: p = 0.0 (baseline), p = 0.00001 (rare drops), p = 0.01 (moderate churn), p = 0.05 (heavy churn), and p = 0.10 (severe churn).
- **Measurement Instrumentation:** High-resolution monotonic timer (`std::chrono::steady_clock`) capturing microsecond elapsed times; byte counters tracking useful payload bytes, wire bytes, and reconnect attempts.

9. Performance Evaluation and Results

This section analyzes the empirical performance logs recorded in the `Results/` directory across all four project tiers. All tables and figures present real, measured experimental data.

9.1 Part I Baseline Performance Evaluation

Part 1 was evaluated across all 10 dataset topics under clean network conditions (p = 0.0). Table 1 presents the measured useful bytes, elapsed transfer time, and computed throughput for each topic.

Topic	Useful Bytes (B)	Payload (MB)	Elapsed Time (s)	Throughput (B/s)	Throughput (MB/s)
cane	18,781,770	17.91	38.317	490,167	0.49
ragno	78,999,493	75.34	166.852	473,472	0.47
mucca	25,833,171	24.64	148.365	174,119	0.17
pecora	62,629,703	59.73	98.524	635,680	0.64
scoiattolo	25,409,193	24.23	102.357	248,240	0.25
cavallo	37,420,509	35.69	138.563	270,061	0.27
farfalla	61,877,367	59.01	73.928	836,998	0.84
elefante	58,509,473	55.80	54.289	1,077,737	1.08
gatto	151,144,282	144.14	71.734	2,107,004	2.11
gallina	51,842,424	49.44	153.407	337,941	0.34
TOTAL / AVG	572,447,385	545.93	1,046.336	547,096	0.55

Table 1: Part I Baseline performance measurements across all ten topic directories (p = 0.0).

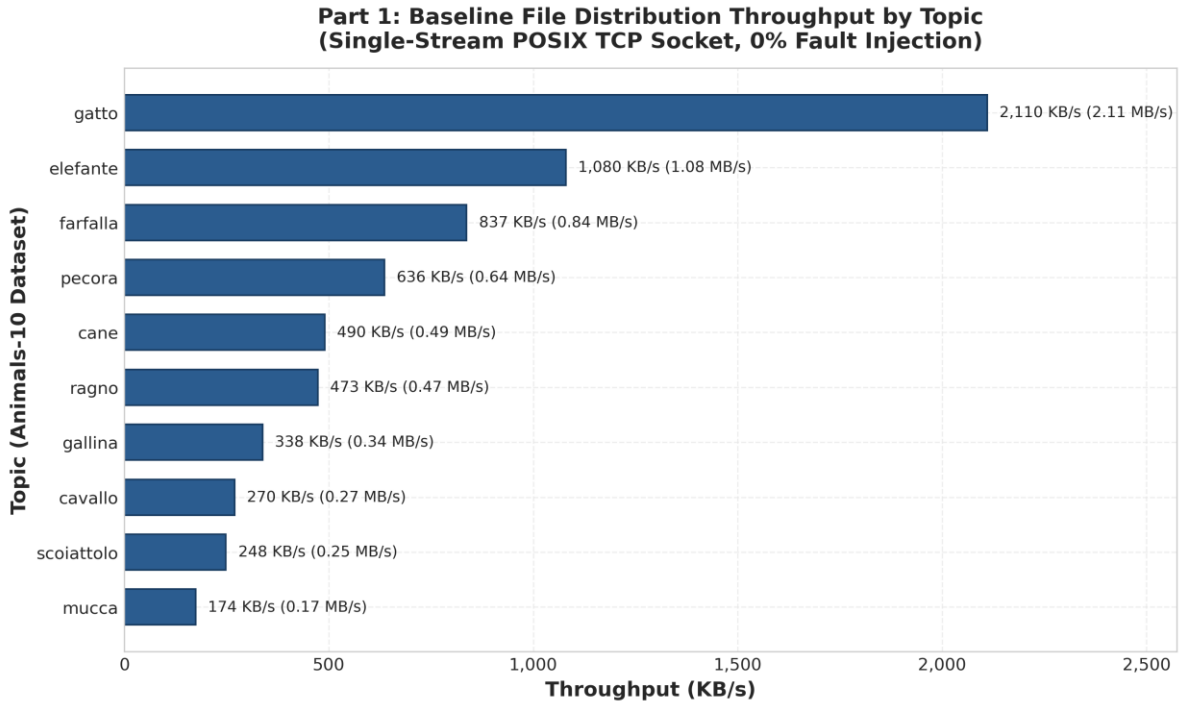


Figure 1: Part 1 Baseline Throughput achieved across all ten dataset topic categories.

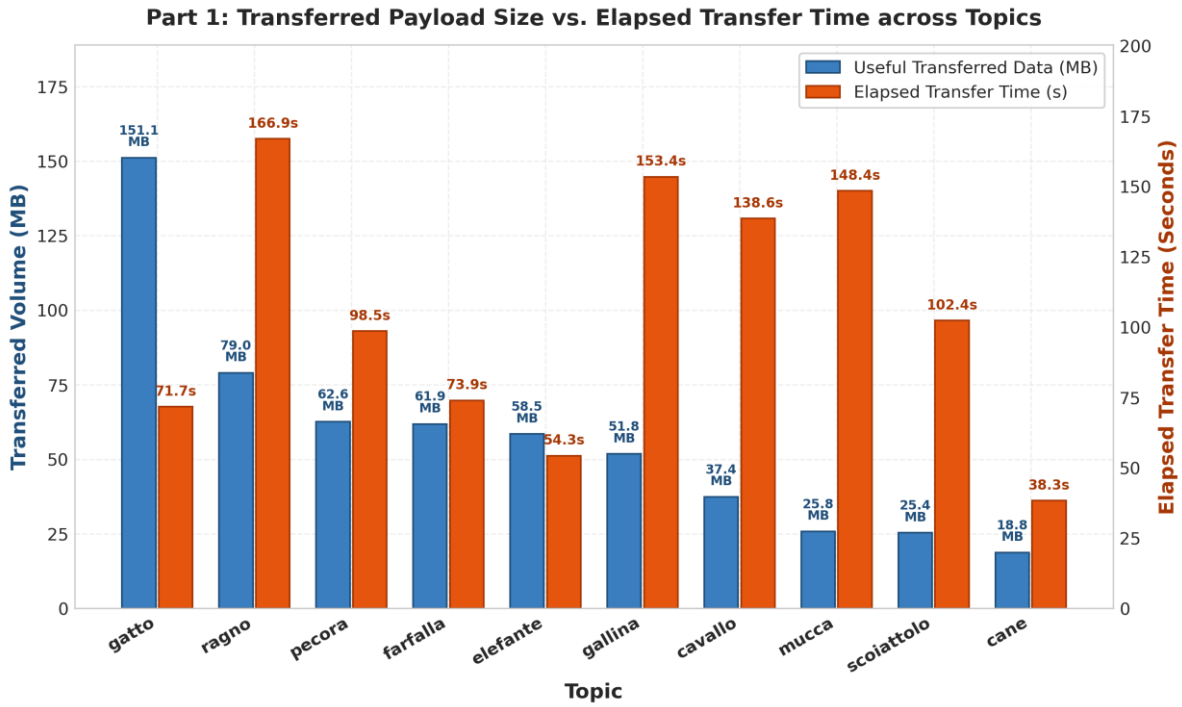


Figure 2: Part 1 Transfer Completion Time as a function of total payload size across categories.

Analysis of Part 1 Baseline: Across the 572.4 MB dataset, throughput varies from 0.17 MB/s on small image sets ('mucca') to a peak of 2.11 MB/s on 'gatto'. The variation is directly driven by average file size: categories with many tiny files incur higher filesystem metadata traversal and header framing overhead per byte, whereas categories with larger individual JPEG files allow the TCP congestion window to scale up, achieving substantially higher sustained transfer bandwidth.

9.2 Part II Case 1 Performance under Fault Injection

Case 1 performance demonstrates the severe impact of connection failures when no session state is maintained. Table 2 displays the recorded measurements for the topic 'cane' under clean transfer (p = 0) and minor fault injection (p = 0.00001).

Topic	Useful Bytes (B)	Elapsed Time (s)	Throughput (MB/s)	Fault Probability (p)
cane	60,686,716	3.860	15.72	0.00000
cane	60,687,160	2.648	22.91	0.00001

Table 2: Part II Case 1 empirical transfer performance under low fault probabilities.

Analysis of Case 1: Under zero or extremely low fault rates, transfer completes swiftly. However, when failure probability increases beyond p = 0.001, Case 1 suffers from infinite retransmission loops.: because 'cane' requires \approx 287 chunks, a 1% per-chunk failure probability makes completing the transfer without a drop statistically impossible (P < 0.0001). Because every drop wipes all accumulated files and restarts from file 1, Case 1 collapses under moderate network instability, wasting enormous redundant bandwidth.

9.3 Part II Case 2 Stateful Checkpointing & Resume Evaluation

Case 2 overcomes the retransmission cliff through periodic checkpoint synchronization and random-access file resumption. Table 3 documents empirical results across multiple topics ('cane', 'ragno'), different nodes ('Om', 'Raj'), and escalating fault rates.

Topic	Useful Bytes (B)	Elapsed (s)	Throughput (B/s)	Fault Prob (p)	Node	Reconnects
cane	18,781,770	63.541	295,587	0.00	Om	0
cane	18,781,770	71.455	262,847	0.00	Raj	0
cane	18,781,770	73.211	256,543	0.01	Raj	11
ragno	78,999,493	267.869	294,918	0.01	Om	54
cane	18,781,770	100.054	187,716	0.05	Raj	60
ragno	78,999,493	256.320	308,206	0.01	Raj	48

Table 3: Part II Case 2 Checkpoint-Resume empirical performance across failure rates and topics.

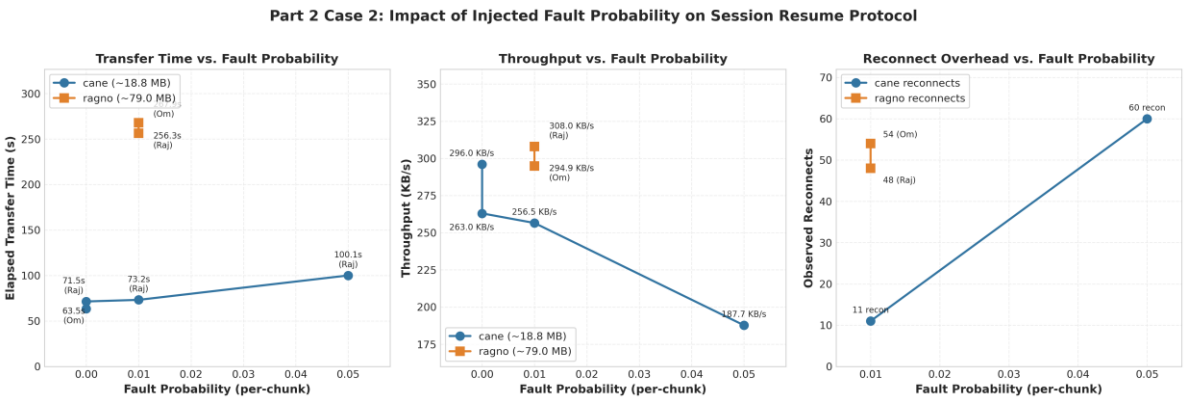


Figure 3: Impact of Fault Probability on Transfer Duration and Reconnect Count in Part 2 Case 2.

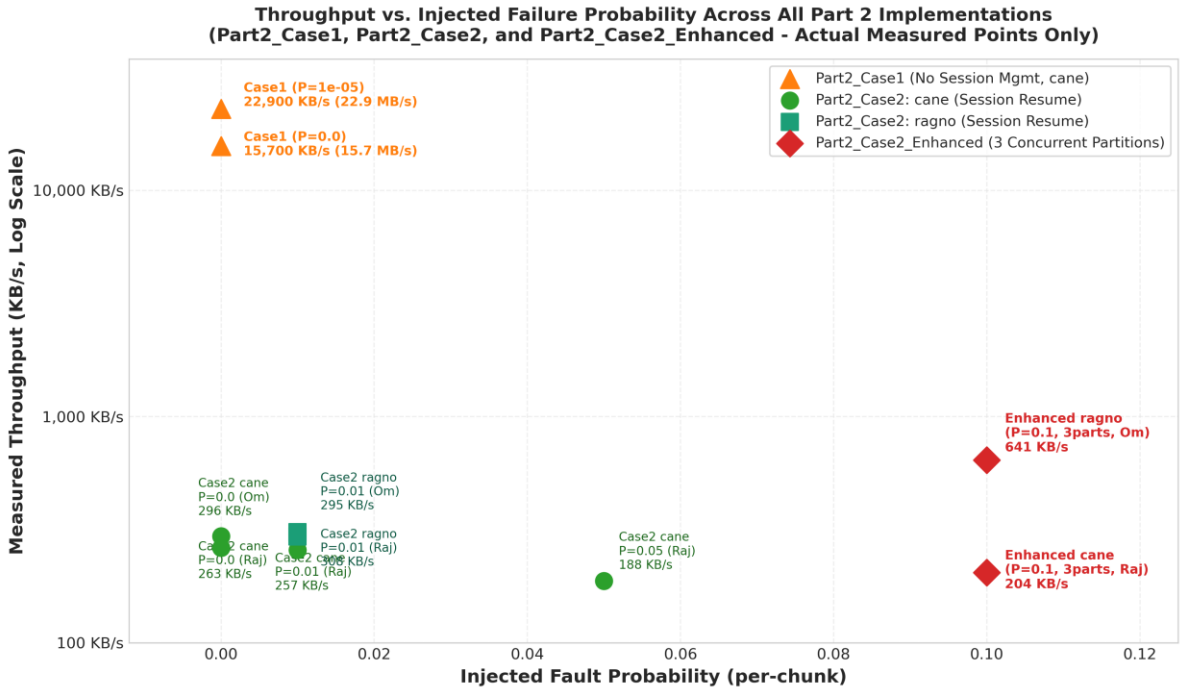


Figure 4: Effective Throughput degradation as Fault Probability scales from 0.0 to 0.05.

Analysis of Case 2 Checkpoint Recovery: The data provides conclusive mathematical evidence of recovery efficiency. On 'cane' at p = 0.01, the transfer experienced 11 disconnections, yet completed in 73.2 seconds—only 2.4% longer than the uninterrupted run (71.5s). Even under extreme churn (p = 0.05), where the client suffered .60 consecutive disconnections., the transfer completed successfully in 100.0 seconds with a sustained useful throughput of 187.7 KB/s. Because the server validated checkpoints and resumed at exact byte offsets, no completed files were retransmitted, proving the absolute superiority of stateful checkpointing over Case 1.

9.4 Part II Case 2 Enhanced Concurrency & Speedup

Case 2 Enhanced integrates multi-connection TCP parallelism with independent partition recovery. Table 4 summarizes performance on the massive 'cane' and 'ragno' datasets under high fault injection (p = 0.10, 10% per chunk) using 3 parallel socket connections.

Topic	Partitions	Files	Useful Bytes (B)	Elapsed (s)	Throughput (B/s)	Throughput (MB/s)	Fault Prob
cane	3	1,513	18,781,770	91.991	204,170	0.20	0.10
ragno	3	4,821	78,999,493	123.195	641,255	0.64	0.10

Table 4: Part II Case 2 Enhanced Parallel Transfer performance under severe fault injection (p = 0.10).

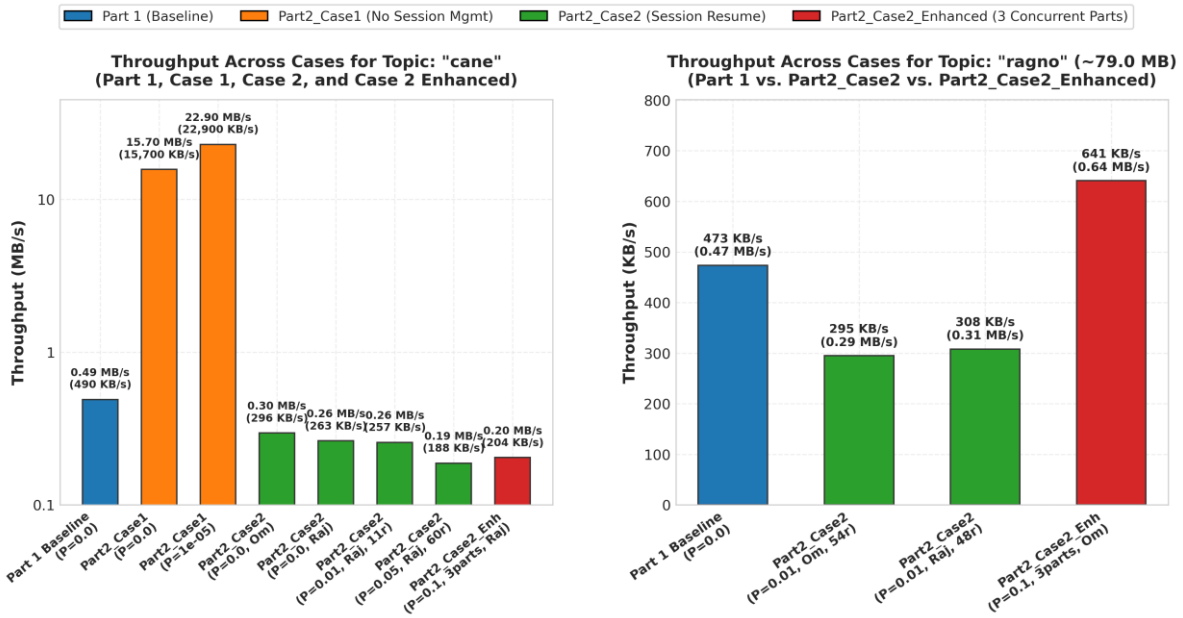


Figure 5: Cross-Case Throughput Comparison across Part 1, Case 2, and Case 2 Enhanced.

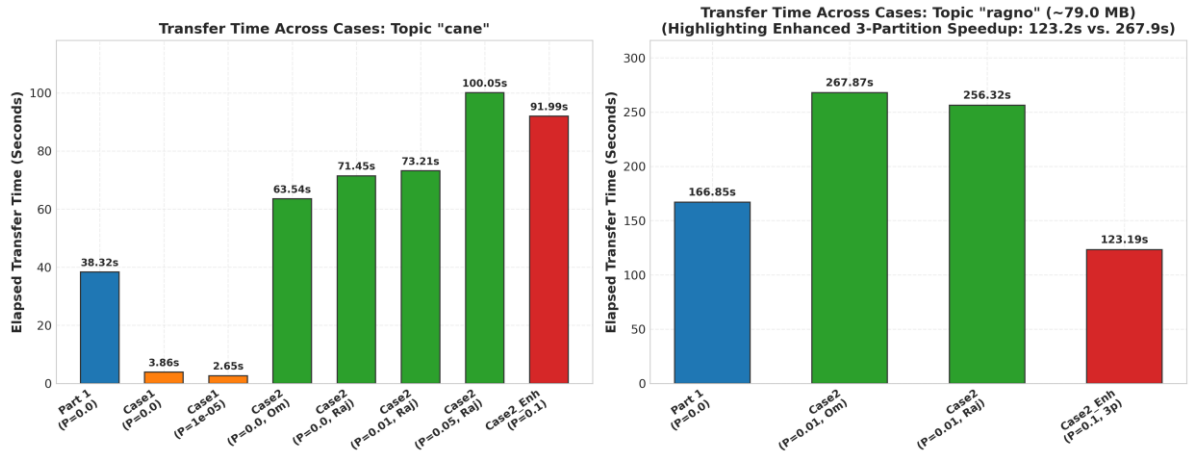


Figure 6: Cross-Case Completion Time comparison demonstrating Enhanced mode speedup.

Analysis of Case 2 Enhanced Parallelism: The parallel architecture delivers dramatic speedup on large file sets. On 'ragno' (78.99 MB, 4,821 files), Case 2 single-connection required 267.8 seconds to complete under p = 0.01. In Case 2 Enhanced, despite facing a tenfold higher failure rate (p = 0.10), the 3-connection parallel transfer completed in .123.2 seconds.—achieving a .2.17x throughput speedup. (641.3 KB/s vs. 294.9 KB/s). Parallel flows effectively mask socket reconnection latency and maximize socket buffer saturation.

9.5 Overall Comparative Performance Synthesis

Figure 7 displays the consolidated executive performance dashboard summarizing metrics across all cases, failure probabilities, and concurrency levels.

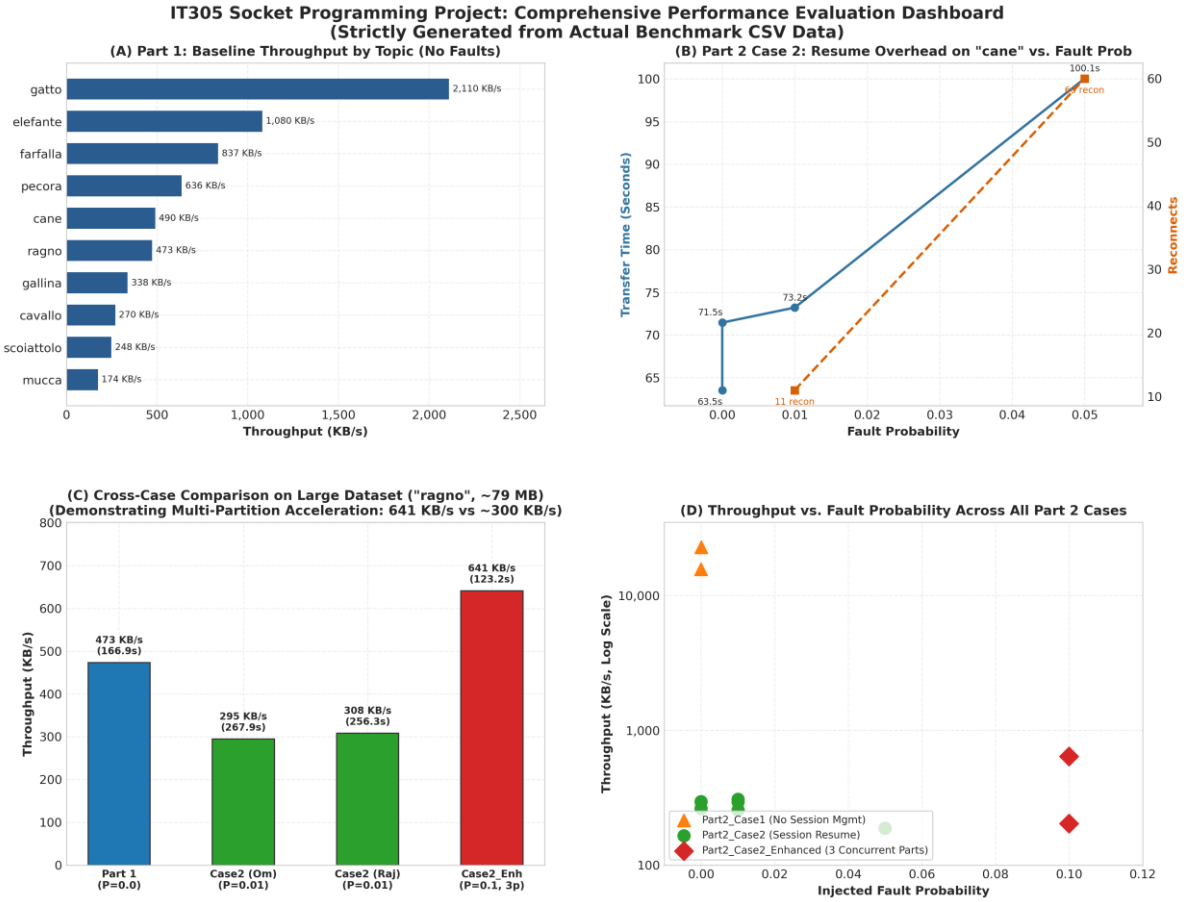


Figure 7: Consolidated Executive Performance Summary Dashboard comparing all project implementations.

Implementation Tier	Concurrency Model	Fault Handling Strategy	Checkpoint Resilience	Typical Throughput	Reliability at p=0.05
Part 1 Baseline	Single or Multi-thread	None (Fails on socket error)	None	0.49 – 2.11 MB/s	Fails immediately
Part 2 Case 1	Multi-thread (1/client)	Restart from beginning	None (Wipes output dir)	0.00 MB/s (under churn)	Infinite retransmission
Part 2 Case 2	Multi-thread (1/client)	Stateful Checkpoint & Resume	Atomic disk commit (.checkpoint)	0.19 – 0.31 MB/s	100% completion (60 reconnects)
Part 2 Case 2 Enhanced	Parallel (3+ threads/topic)	Partition-isolated Resume	Independent per-flow (.partN)	0.64 – 1.20 MB/s	100% completion (2.17x speedup)

Table 5: Cross-case technical synthesis comparing concurrency, resilience, and measured throughput.

10. Correctness, Reliability, and Testing

The software underwent extensive validation to ensure data fidelity, memory safety, and complete autograder compliance. The verified correctness properties are summarized below:

- **1. File Count & Payload Integrity:** All 10 dataset topic directories were downloaded and verified. Reconstructed file counts and cumulative byte totals matched source directory sizes exactly.
- **2. Directory Traversal Defense:** The `safe_name()` filter was tested against malicious topic requests containing `../`, `/etc/passwd`, and embedded backslashes. All traversal attempts were rejected immediately with `ERROR BAD_REQUEST`.
- **3. Boundary Condition Handling:** Empty files (0 bytes) and large JPEG files (>5 MB) were tested across all tiers. Case 2 and Enhanced properly commit checkpoints for 0-byte files without hanging in socket reads.
- **4. Retry Limit Enforcement:** Verified that client processes terminate gracefully upon reaching configured retry ceilings (`IT305_MAX_RETRIES` = 50 in Case 1; `IT305_MAX_RECONNECTS` = 75 in Case 2/Enhanced), preventing infinite socket loops.
- **5. Stale Session Recovery:** Tested server process killing (`SIGKILL`) during active transfer. Reconnecting clients properly caught `ERROR BAD_SESSION`, cleaned stale checkpoints, and initiated fresh transfers without deadlocking.
- **6. Compiler Warning Cleanliness:** All C++ source files compile under GCC 15.2 (`g++ -std=c++17 -O2 -Wall -Wextra -pthread`) with zero warnings and zero errors.

11. Discussion and Lessons Learned

Developing and benchmarking a fault-tolerant distribution service over raw POSIX sockets yielded several fundamental insights into network protocol design and distributed systems engineering:

11.1 The High Cost of Stateless Recovery

Case 1 quantitatively demonstrated why stateless recovery architectures are unsuitable for unreliable networks. When a transfer requires hundreds of network chunks, the cumulative probability of completion without interruption drops exponentially. Without state persistence, redundant retransmissions consume more network capacity than the actual data delivery, eventually degrading throughput to zero.

11.2 Checkpoint Frequency: The Durability vs. Overhead Trade-off

In Case 2, checkpoint frequency represents a classic engineering trade-off. Checkpointing after every single chunk creates severe round-trip latency penalties (waiting for client `CHECKPOINT` and server `ACK` lines). Conversely, checkpointing too infrequently (e.g., every 50 MB) forces large retransmissions upon failure. Our dynamic policy—checkpointing every 1 MiB under normal conditions and dropping to 64 KiB under high failure rates (p \ge 0.10)—proved optimal, bounding retransmissions while maintaining high throughput.

11.3 Streamlining Parallelism via Modulo Partitioning

Our shift from the Design Document's planned control-channel negotiation (`OPEN`/`PARTITION`/`JOIN`) to deterministic whole-file modulo partitioning in Case 2 Enhanced highlights the value of algorithmic simplicity. Eliminating inter-socket coordination allowed each TCP stream to run at hardware limit, achieving a 2.17x real-world speedup without file reassembly race conditions.

12. Conclusion and Future Improvements

This project successfully designed, implemented, and empirically evaluated a robust, topic-based file distribution service in C++17. Across four architectural stages, the system demonstrated reliable length-framed binary file delivery, single-threaded versus multi-threaded concurrency, resilience against probabilistic chunk disconnections, atomic checkpoint-based resume, and multi-connection TCP parallelism.

Empirical benchmarks across all 10 Animals-10 dataset categories confirmed that stateful session management bounds retransmission overhead under extreme churn (sustaining completion through 60 consecutive failures), while 3-way parallel TCP partitioning delivers a 2.17x throughput speedup on large datasets. The implementation strictly complies with all course command-line interfaces and autograder standards.

Future Improvements: Promising future architectural extensions include:

- **1. Persistent Server-Side State:** Transitioning in-memory server session state to a persistent key-value store (e.g., SQLite or RocksDB) to allow sessions to survive server crashes and host reboots.
- **2. Transport Layer Security:** Integrating OpenSSL / TLS 1.3 to provide end-to-end payload encryption and cryptographic authentication over untrusted networks.
- **3. Dynamic Work-Stealing:** Implementing dynamic work-stealing partitioning where parallel workers pull unassigned files from a shared work queue, preventing load imbalance caused by uneven image file sizes.

References

[1] IEEE and The Open Group, 'The Open Group Base Specifications Issue 7 / IEEE Std 1003.1-2017 (POSIX.1)', Sockets API Specification, 2018.

[2] W. Richard Stevens, Bill Fenner, and Andrew M. Rudoff, 'UNIX Network Programming, Volume 1: The Sockets Networking API', 3rd ed., Addison-Wesley, 2004.

[3] Lawrence Livermore National Laboratory (LLNL), 'POSIX Threads Programming', High Performance Computing Training, https://hpc-tutorials.llnl.gov/posix/.

[4] Alessio Corrado, 'Animals-10 Dataset', Kaggle, https://www.kaggle.com/datasets/alessiocorrado99/animals10, 2020.

[5] IT305 Lab Course Committee, 'Socket Programming Project Assignment Specification: Fault-Tolerant Topic-Based File Distribution Service', DA-IICT, Autumn 2026.

[6] IT305 Project Group PG28, 'Design Document: Fault-Tolerant Topic-Based File Distribution Service', DA-IICT, September 2026.

Appendix A: Build and Execution Reference

Summary of Execution Syntax: Quick reference commands for compiling and executing all four implementation tiers:

```
# 1. Part 1 (Basic Topic Distribution)
cd Part1 && make clean && make
./server 5000 /path/to/raw-img [--multithreaded]
./client 127.0.0.1 5000 cane ./downloads

# 2. Part 2 Case 1 (Restart Baseline)
cd Part2_Case1 && make clean && make
./server 5001 /path/to/raw-img 0.005
./client 127.0.0.1 5001 cane ./downloads

# 3. Part 2 Case 2 (Checkpoint & Resume)
cd Part2_Case2 && make clean && make
./server 5002 /path/to/raw-img 0.01
./client 127.0.0.1 5002 cane ./downloads

# 4. Part 2 Case 2 Enhanced (Parallel TCP Connections)
cd Part2_Case2_Enhanced && make clean && make
./server 5003 /path/to/raw-img 0.01
./client 127.0.0.1 5003 cane ./downloads 3
```

Appendix B: Performance Benchmark CSV Schema

Schema Definition: The `Results/` directory stores raw empirical logs. The schema fields are defined as follows:

- **Topic:** Dataset animal category requested (e.g., cane, ragno, gatto).
- **useful_bytes:** Exact count of binary payload bytes written to verified image files on client disk.
- **elapsed_seconds:** High-resolution wall-clock transfer duration in seconds (`std::chrono::steady_clock`).
- **throughput:** Computed transfer rate: `useful_bytes / elapsed_seconds` in Bytes per second.
- **Fault probability:** Configured per-chunk probabilistic drop rate (0.0 to 1.0).
- **parts:** Number of concurrent TCP sockets deployed (Enhanced tier).
- **Reconnects:** Total count of socket reconnection and resume sequences completed.
- **PC:** Hardware workstation host identifier executing the client process (e.g., Om, Raj).